



Inputs de usuario

Un prompt dinámico no solo depende del texto que resume o traduce: depende de **quién usa la app** y **cómo quiere que sea la respuesta** — idioma, tono, longitud, objetivo del mensaje, tipo de tarea.

En Python esos valores suelen venir de:

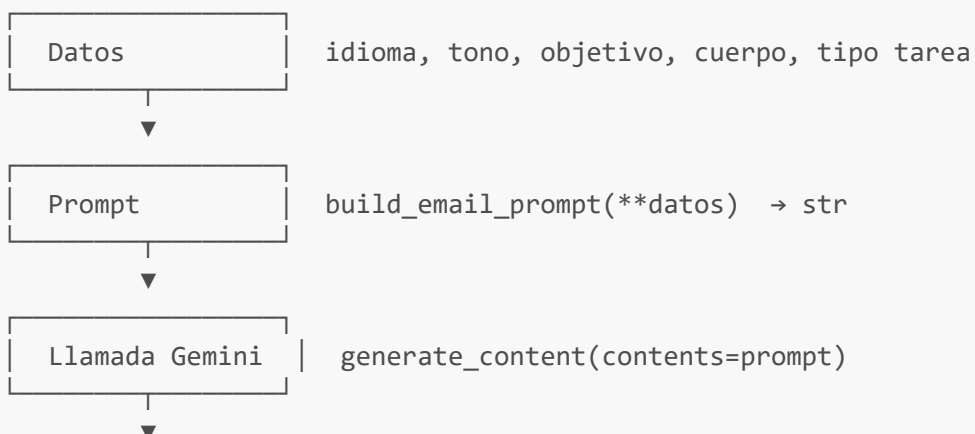
- `input()` en consola,
- formularios (web),
- argumentos de línea de comandos,
- configuración guardada en un `dict` o fichero.

La habilidad de esta sección: **separar datos del usuario, plantilla del prompt y llamada a Gemini**.

Objetivos

- Parametrizar prompts con **preferencias de usuario** (idioma, tono, objetivo).
 - Mantener **datos / prompt / llamada** en capas distintas.
 - Validar inputs antes de construir el prompt (Unidad 1).
 - Montar un flujo tipo “generador de emails” en Python + Gemini.
-

1) Tres capas (repetir hasta automatizarlo)



Salida

print / guardar / parsear JSON

Si mezclas las tres en un solo bloque sin funciones, no podrás reutilizar el prompt en otras tareas sin tener que modificar el código.

2) Configuración de usuario como `dict`

```
user_prefs = {
    "idioma": "español",
    "tono": "formal",          # formal | cercano | directo
    "objetivo": "solicitar una reunión",
    "destinatario": "equipo de producto",
}

cuerpo_usuario = """
Necesito coordinar una demo del sprint el viernes.
¿Podéis confirmar disponibilidad?
"""
```

Whitelist de tonos (validación):

```
TONOS_VALIDOS = {"formal", "cercano", "directo"}
IDIOMAS_VALIDOS = {"español", "inglés", "francés"}

def validar_prefs(prefs: dict) -> list[str]:
    errores = []
    if prefs.get("tono") not in TONOS_VALIDOS:
        errores.append(f"tono debe ser uno de: {TONOS_VALIDOS}")
    if prefs.get("idioma") not in IDIOMAS_VALIDOS:
        errores.append(f"idioma debe ser uno de: {IDIOMAS_VALIDOS}")
    if not str(prefs.get("objetivo", "")).strip():
        errores.append("objetivo no puede estar vacío")
    return errores
```

3) Plantilla de email parametrizada

```
PLANTILLA_EMAIL = """
Eres un asistente de redacción profesional.

Tarea: redacta un email completo (asunto + cuerpo) listo para enviar.

Parámetros:
```

- Idioma: {idioma}
- Tono: {tono}
- Objetivo del email: {objetivo}
- Destinatario: {destinatario}

Instrucciones:

- Incluye línea de asunto al inicio (prefijo "Asunto: ").
- Máximo 180 palabras en el cuerpo.
- No inventes nombres de personas ni fechas que el usuario no haya indicado.

Borrador o notas del usuario:

```
{notas}
"""
```

```
def build_email_prompt(prefs: dict, notas: str) -> str:
    errores = validar_prefs(prefs)
    if errores:
        raise ValueError("; ".join(errores))
    return PLANTILLA_EMAIL.format(
        idioma=prefs["idioma"],
        tono=prefs["tono"],
        objetivo=prefs["objetivo"],
        destinatario=prefs.get("destinatario", "destinatario genérico"),
        notas=notas.strip() or "(sin notas adicionales)",
    )
```

4) Recoger inputs por consola

```
def pedir_prefs() -> dict:
    print("Configura el email:")
    return {
        "idioma": input("Idioma (español/inglés/francés): ").strip().lower() or
"español",
        "tono": input("Tono (formal/cercano/directo): ").strip().lower() or
"formal",
        "objetivo": input("Objetivo del email: ").strip(),
        "destinatario": input("Destinatario: ").strip(),
    }

def main():
    prefs = pedir_prefs()
    notas = input("\nNotas o borrador:\n")
    prompt = build_email_prompt(prefs, notas)

    from google import genai
    client = genai.Client()
    response = client.models.generate_content(
        model="gemini-3-flash-preview",
```

```

        contents=prompt,
    )
    print("\n", response.text)

```

En producto real, `pedir_prefs()` sería un formulario; la **plantilla** no cambia.

5) Inputs que cambian la tarea (tipo de ticket)

Relación con Unidad 1:

```

def pedir_registro() -> dict:
    return {
        "nombre": input("Nombre: ").strip(),
        "tipo": input("Tipo (summary/translate/qa): ").strip().lower(),
        "texto": input("Texto o pregunta:\n"),
    }

```

```

def prompt_desde_registro(reg: dict, roles: dict, tasks: dict) -> str:
    tipo = reg["tipo"]
    role = roles.get("teacher", "")
    task = tasks[tipo]
    return build_prompt(role, task, reg["texto"])

```

El **usuario** elige el tipo; Python selecciona la **tarea** correcta.

6) Perfiles guardados (sin base de datos)

```

PERFILES = {
    "estudiante": {"idioma": "español", "tono": "cercano"},
    "empresa": {"idioma": "español", "tono": "formal"},
}

def prefs_desde_perfil(perfil_id: str, overrides: dict | None = None) -> dict:
    if perfil_id not in PERFILES:
        raise ValueError(f"Perfil desconocido: {perfil_id}")
    base = {**PERFILES[perfil_id]}
    if overrides:
        base.update(overrides)
    return base

```

El prompt se adapta al perfil **sin** que el usuario reescriba el rol cada vez. Si el usuario es una empresa, el prompt se adaptará a su tono formal, si es un estudiante, se adaptará a su tono cercano.

7) Separar archivo `prompts.py` y `main.py`

`prompts.py`

- `PLANTILLA_EMAIL`, `build_email_prompt`, `validar_prefs`

`main.py`

- `pedir_prefs`, llamada Gemini, `print`

`config.py`

- `MODEL`, listas `TONOS_VALIDOS`

Así cambias textos sin tocar el flujo de `input()`.

8) Respuesta estructurada de la app (Unidad 1)

Después de Gemini, puedes envolver igual que en Unidad 1:

```
def respuesta_ok(mensaje: str, data: dict | None = None) -> dict:
    return {"status": "ok", "mensaje": mensaje, "data": data or {}}

def generar_email(prefs: dict, notas: str) -> dict:
    errores = validar_prefs(prefs)
    if errores:
        return {"status": "error", "mensaje": "Prefs inválidas", "data": {"errores": errores}}

    prompt = build_email_prompt(prefs, notas)
    texto = llamar_gemini(prompt)
    return respuesta_ok("Email generado", {"email": texto, "prefs": prefs})
```

El LLM devuelve texto libre; tu app devuelve **dict** al frontend o al `main`.

9) Buenas prácticas con inputs de usuario

1. **Validar antes** de llamar a la API (ahorro de coste y errores).
 2. **Valores por defecto sensatos** (`idioma="español"`) pero documentados.
 3. **No confiar** en que el usuario escriba bien el tono — whitelist o menú numérico.
 4. **Normalizar** (`strip()`, `lower()` en claves internas).
 5. **Mostrar preview del prompt** en modo desarrollo.
-

10) Errores frecuentes

Error	Consecuencia
Pasar el input crudo sin validar	Prompts rotos o respuestas basura
Mezclar prefs y prompt en <code>main</code>	Imposible reutilizar ni testear
Olvidar indicar idioma en la plantilla	Respuesta en inglés inesperada
Re-pedir al LLM lo que ya validaste en Python	Gasto innecesario

Resumen

- Los prompts **se adaptan al usuario** vía variables y plantillas.
- Patrón: **datos** → `build_*_prompt` → **Gemini** → **salida**.
- Validación Python previa; plantillas en `prompts.py`.
- El generador de emails es el caso canónico de esta sección.